09 — Evaluación y Comparación de Modelos

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook evalúa los cinco modelos con las **predicciones out-of-fold (OOF)** de la validación cruzada del notebook 08 sobre las **8274 imágenes**. No hay hold-out ni test separado: la estimación es la CV OOF.

Métricas

- Accuracy, balanced accuracy, precision, recall y F1 (macro y weighted).
- Métricas por clase (`health` , `sick` , `tb`).
- AUC-ROC por clase (One-vs-Rest) y **Average Precision (AP)** de `tb` .
- Intervalos de confianza por bootstrap y McNemar con Holm.
- Calibración de `tb` (Brier + curva).

Protocolo

1. Cargar las predicciones y scores OOF del notebook 08.
2. Evaluar las 8274 imágenes (cada una predicha exactamente una vez).
3. Comparar modelos y analizar la incertidumbre.
4. Recomendar el modelo elegido por la CV del 08.

Flujo

```
data/08_oof_predicciones.csv
data/08_oof_scores.npz
data/08_manifest.json
        |
        v
 Metricas globales + por clase + AUC/AP
        |
        v
 Matrices de confusion + curvas ROC/AP
        |
        v
 Bootstrap + McNemar (global y tb) + calibracion
        |
        v
 Recomendacion final
```

0. Configuración del entorno

In [1]:
```
# Se configura el path del proyecto y se importan las librerías de evaluación.
# Referencias:
#   metricas: https://scikit-learn.org/stable/modules/model_evaluation.html
#   calibracion: https://scikit-learn.org/stable/modules/calibration.html

import sys, os, warnings, json
warnings.filterwarnings('ignore')

from pathlib import Path
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from scipy.stats import chi2

from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    ConfusionMatrixDisplay,
    confusion_matrix,
    roc_curve,
    auc,
    precision_recall_curve,
    average_precision_score,
    roc_auc_score,
    brier_score_loss,
)
from sklearn.calibration import calibration_curve

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.visualization import set_publication_style

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

CLASS_NAMES = ['health', 'sick', 'tb']
N_BOOTSTRAP = 1000

DATA_DIR    = Path('..') / 'data'
REPORTS_DIR = Path('..') / 'reports'
REPORTS_DIR.mkdir(exist_ok=True)

set_publication_style()
print('Entorno configurado correctamente.')
```

```
Entorno configurado correctamente.
```

1. Carga de las predicciones out-of-fold

In [2]:

```
# Se cargan las predicciones y scores OOF del notebook 08.
# Cada imagen fue predicha exactamente una vez por el modelo de su fold externo.
# Referencia: https://numpy.org/doc/stable/reference/generated/numpy.load.html

for archivo in ['08_oof_predicciones.csv', '08_oof_scores.npz', '08_manifest.json']:
    if not (DATA_DIR / archivo).exists():
        raise FileNotFoundError(
            f'No se encontró {archivo}. Ejecuta primero el notebook 08.'
        )

oof = pd.read_csv(DATA_DIR / '08_oof_predicciones.csv')
score_data = np.load(DATA_DIR / '08_oof_scores.npz')

y_true = oof['y_true'].to_numpy()
MODELOS = [c for c in oof.columns if c not in ('y_true', 'Dummy')]
predicciones = {n: oof[n].to_numpy() for n in MODELOS}
puntuaciones = {n: score_data[n] for n in MODELOS}

with open(DATA_DIR / '08_manifest.json', encoding='utf-8') as f:
    manifest = json.load(f)

print(f'Imágenes OOF : {len(y_true)}')
print('Distribución:')
for clase, conteo in zip(*np.unique(y_true, return_counts=True)):
    print(f'  {CLASS_NAMES[clase]:6s} (label={clase}): {conteo} muestras')
print()
```

```
print('Modelos:', MODELOS)
print(f'Modelo seleccionado por CV: {manifest.get("seleccionado_cv")}')
print(f'Estrategia : {manifest.get("estrategia")}')
print(f'Selector   : {manifest.get("selector")}')
```

```
Imágenes OOF : 8274
Distribución:
  health (label=0): 3800 muestras
  sick   (label=1): 3674 muestras
  tb     (label=2): 800 muestras

Modelos: ['Logistic Regression', 'SVM', 'Decision Tree', 'Random Forest', 'XGBoost']
Modelo seleccionado por CV: SVM
Estrategia : class_weight
Selector   : {'metodo': 'RF', 'n_features': 1000}
```

2. Comparación global

In [3]:

```
# Métricas globales sobre las 8274 predicciones OOF.
# AP = Average Precision (average_precision_score); no es el área trapezoidal de la PR.

filas = []
for nombre in MODELOS:
    y_pred = predicciones[nombre]
    y_score = puntuaciones[nombre]
    filas.append({
        'Modelo': nombre,
        'Accuracy': accuracy_score(y_true, y_pred),
        'Balanced Acc': balanced_accuracy_score(y_true, y_pred),
        'Precision macro': precision_score(y_true, y_pred, average='macro', zero_division=0),
        'Recall macro': recall_score(y_true, y_pred, average='macro', zero_division=0),
        'F1 macro': f1_score(y_true, y_pred, average='macro', zero_division=0),
        'Precision weighted': precision_score(y_true, y_pred, average='weighted', zero_division=0),
        'Recall weighted': recall_score(y_true, y_pred, average='weighted', zero_division=0),
        'F1 weighted': f1_score(y_true, y_pred, average='weighted', zero_division=0),
        'Recall tb': recall_score(y_true, y_pred, labels=[2], average=None, zero_division=0)[0],
        'AUC-ROC macro': np.mean([
            roc_auc_score((y_true == c).astype(int), y_score[:, c])
            for c in range(len(CLASS_NAMES))
        ]),
        'AP tb': average_precision_score((y_true == 2).astype(int), y_score[:, 2]),
    })

resultados_globales = pd.DataFrame(filas).sort_values(
    ['Balanced Acc', 'F1 macro'], ascending=False,
).reset_index(drop=True)

print('Comparación global out-of-fold (8274 imágenes):')
print(resultados_globales.round(4).to_string(index=False))

resultados_globales.to_csv(DATA_DIR / '09_resultados_globales.csv', index=False)
print()
print(f'Guardado en: {(DATA_DIR / "09_resultados_globales.csv").resolve()}')
```

```
Comparación global out-of-fold (8274 imágenes):
             Modelo  Accuracy  Balanced Acc  Precision macro  Recall macro  F1 macro  Precision weighted
Recall weighted  F1 weighted  Recall tb  AUC-ROC macro  AP tb
                SVM    0.9689        0.9494           0.9369        0.9494    0.9429              0.9697
0.9689       0.9692     0.8950         0.9871 0.8929
            XGBoost    0.9722        0.9476           0.9544        0.9476    0.9509              0.9720
0.9722       0.9721     0.8788         0.9964 0.9596
Logistic Regression    0.9633        0.9462           0.9259        0.9462    0.9353              0.9648
0.9633       0.9638     0.8988         0.9942 0.9367
      Random Forest    0.9485        0.9085           0.9175        0.9085    0.9128              0.9480
0.9485       0.9482     0.7962         0.9903 0.8994
      Decision Tree    0.8843        0.8581           0.8066        0.8581    0.8242              0.9022
0.8843       0.8905     0.7850         0.9350 0.5824

Guardado en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\09_resultados_globales.csv
```

3. Métricas por clase

In [4]:
```
# Precision, recall y F1 por clase, más AUC-ROC One-vs-Rest.

filas = []
for nombre in MODELOS:
    y_pred = predicciones[nombre]
    y_score = puntuaciones[nombre]
    for k, clase in enumerate(CLASS_NAMES):
        filas.append({
            'Modelo': nombre,
            'Clase': clase,
            'Precision': precision_score(y_true, y_pred, labels=[k], average=None, zero_division=0)[0],
            'Recall': recall_score(y_true, y_pred, labels=[k], average=None, zero_division=0)[0],
            'F1': f1_score(y_true, y_pred, labels=[k], average=None, zero_division=0)[0],
            'AUC-ROC (OvR)': roc_auc_score((y_true == k).astype(int), y_score[:, k]),
        })

resultados_clase = pd.DataFrame(filas)
print('Métricas por clase:')
print(resultados_clase.round(4).to_string(index=False))
resultados_clase.to_csv(DATA_DIR / '09_resultados_por_clase.csv', index=False)
print()
print(f'Guardado en: {(DATA_DIR / "09_resultados_por_clase.csv").resolve()}')
```

```
Métricas por clase:
             Modelo  Clase  Precision  Recall     F1  AUC-ROC (OvR)
Logistic Regression health     0.9832  0.9832 0.9832         0.9982
Logistic Regression   sick     0.9783  0.9567 0.9674         0.9961
Logistic Regression     tb     0.8161  0.8988 0.8554         0.9884
                SVM health     0.9846  0.9908 0.9877         0.9983
                SVM   sick     0.9817  0.9624 0.9720         0.9953
                SVM     tb     0.8443  0.8950 0.8689         0.9678
      Decision Tree health     0.9487  0.9150 0.9315         0.9642
      Decision Tree   sick     0.9334  0.8743 0.9029         0.9603
      Decision Tree     tb     0.5377  0.7850 0.6382         0.8806
      Random Forest health     0.9623  0.9797 0.9709         0.9962
      Random Forest   sick     0.9585  0.9494 0.9539         0.9930
      Random Forest     tb     0.8316  0.7962 0.8135         0.9815
            XGBoost health     0.9799  0.9897 0.9848         0.9987
            XGBoost   sick     0.9784  0.9744 0.9764         0.9975
            XGBoost     tb     0.9048  0.8788 0.8916         0.9929

Guardado en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\09_resultados_por_clase.csv
```

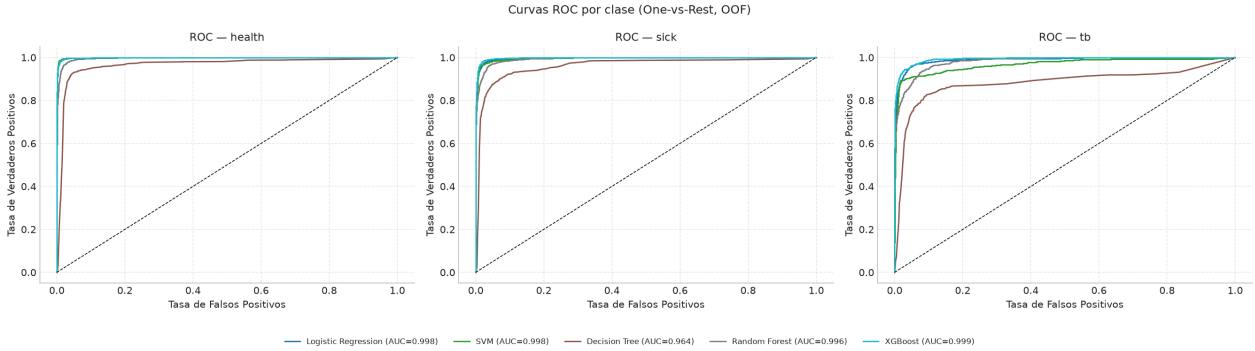
4. AUC-ROC por clase

In [5]:
```
# Curvas ROC One-vs-Rest para las 8274 predicciones OOF.

colores = plt.cm.tab10(np.linspace(0, 1, len(MODELOS)))
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (k, clase) in zip(axes, enumerate(CLASS_NAMES)):
    for color, nombre in zip(colores, MODELOS):
        y_bin = (y_true == k).astype(int)
        fpr, tpr, _ = roc_curve(y_bin, puntuaciones[nombre][:, k])
        ax.plot(fpr, tpr, color=color, lw=1.4,
                label=f'{nombre} (AUC={auc(fpr, tpr):.3f})')
    ax.plot([0, 1], [0, 1], 'k--', lw=0.8)
    ax.set_xlabel('Tasa de Falsos Positivos', fontsize=10)
    ax.set_ylabel('Tasa de Verdaderos Positivos', fontsize=10)
    ax.set_title(f'ROC — {clase}', fontsize=11)
    ax.grid(True, linestyle='--', alpha=0.4)

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc='lower center', ncol=5, fontsize=8, frameon=False)
fig.suptitle('Curvas ROC por clase (One-vs-Rest, OOF)', fontsize=12)
fig.tight_layout(rect=[0, 0.08, 1, 1])
fig.savefig(REPORTS_DIR / '09_roc_por_clase.png', dpi=300, bbox_inches='tight')
plt.show()
```



5. Curvas Average Precision (tb)

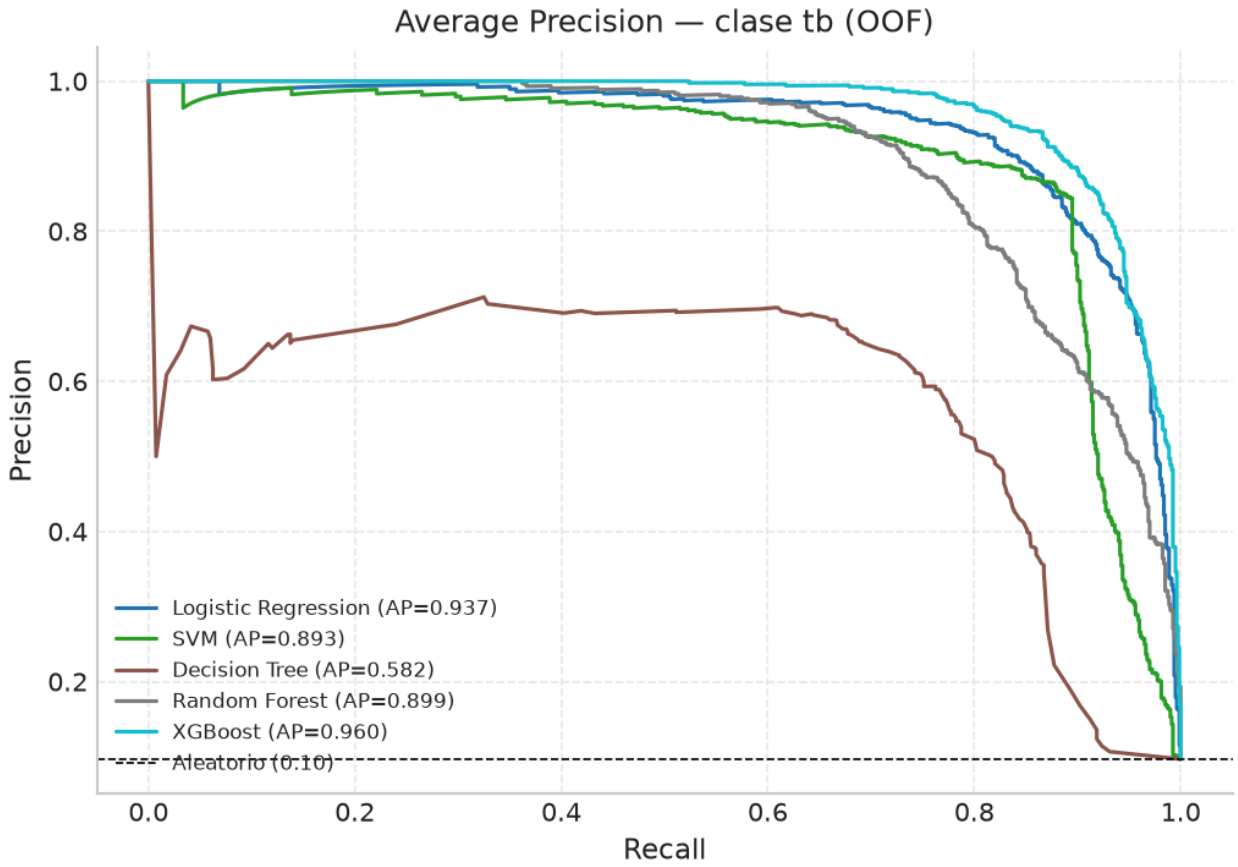
In [6]:

```
# AP (average precision) es más informativa que ROC-AUC con clase positiva minoritaria.
# Es AP, no el área trapezoidal de la curva PR.
# Referencia: https://scikit-
learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html

fig, ax = plt.subplots(figsize=(7, 5))
y_tb = (y_true == 2).astype(int)

for color, nombre in zip(colores, MODELOS):
    precision, recall, _ = precision_recall_curve(y_tb, puntuaciones[nombre][:, 2])
    ap = average_precision_score(y_tb, puntuaciones[nombre][:, 2])
    ax.plot(recall, precision, color=color, lw=1.5,
            label=f'{nombre} (AP={ap:.3f})')

ax.axhline(y_tb.mean(), color='k', linestyle='--', lw=0.8,
           label=f'Aleatorio ({y_tb.mean():.2f})')
ax.set_xlabel('Recall', fontsize=11)
ax.set_ylabel('Precision', fontsize=11)
ax.set_title('Average Precision — clase tb (OOF)', fontsize=12)
ax.legend(fontsize=8)
ax.grid(True, linestyle='--', alpha=0.4)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '09_ap_tb.png', dpi=300, bbox_inches='tight')
plt.show()
```



6. Matrices de confusión y errores

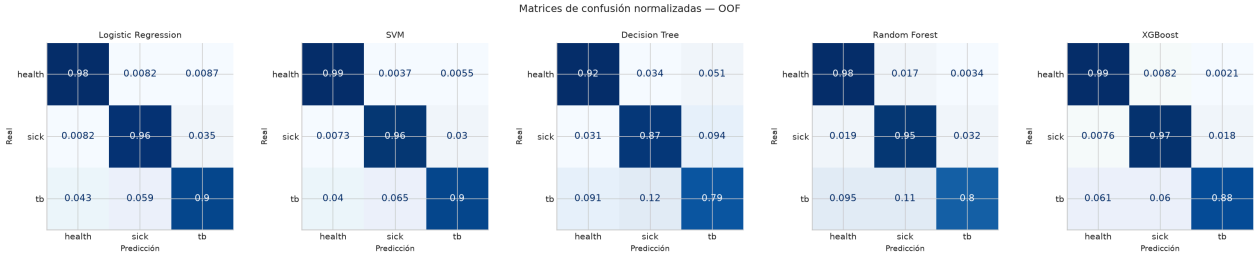
In [7]:

```
# Matrices normalizadas por fila (recall por clase) y conteos de confusiones tb/sick.

fig, axes = plt.subplots(1, len(MODELOS), figsize=(4.2 * len(MODELOS), 4))
for ax, nombre in zip(axes, MODELOS):
    ConfusionMatrixDisplay.from_predictions(
        y_true, predicciones[nombre],
        display_labels=CLASS_NAMES,
        normalize='true', colorbar=False, cmap='Blues', ax=ax,
    )
    ax.set_title(nombre, fontsize=10)
    ax.set_xlabel('Predicción', fontsize=9)
    ax.set_ylabel('Real', fontsize=9)

fig.suptitle('Matrices de confusión normalizadas — OOF', fontsize=12, y=1.02)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '09_matrices_confusion.png', dpi=300, bbox_inches='tight')
plt.show()

print('Confusiones principales (conteos absolutos):')
print(f'{"Modelo":22s}  tb->sick  sick->tb  tb->health  health->tb')
for nombre in MODELOS:
    cm = confusion_matrix(y_true, predicciones[nombre])
    print(f'{nombre:22s}  {cm[2, 1]:7d}  {cm[1, 2]:7d}  {cm[2, 0]:10d}  {cm[0, 2]:9d}')
```



```
Confusiones principales (conteos absolutos):
Modelo                 tb->sick  sick->tb  tb->health  health->tb
Logistic Regression          47       129          34          33
SVM                          52       111          32          21
Decision Tree                99       347          73         193
Random Forest                87       116          76          13
XGBoost                      48        66          49           8
```

7. Comparación estadística

> Los p-valores de McNemar se corrigen con **Holm**. Se reportan el test global y uno restringido a los casos de `tb`.

In [8]:

```python
# Bootstrap estratificado para balanced accuracy y recall de tb; McNemar global y de tb.

def bootstrap_ci(y_true, y_pred, metric_fn, n_boot=N_BOOTSTRAP, seed=RANDOM_STATE):
    rng = np.random.RandomState(seed)
    clases = np.unique(y_true)
    idx = {c: np.where(y_true == c)[0] for c in clases}
    valores = []
    for _ in range(n_boot):
        muestra = np.concatenate([
            rng.choice(idx[c], size=len(idx[c]), replace=True) for c in clases
        ])
        valores.append(metric_fn(y_true[muestra], y_pred[muestra]))
    return np.percentile(valores, [2.5, 97.5])


filas = []
for nombre in MODELOS:
    y_pred = predicciones[nombre]
    bacc_lo, bacc_hi = bootstrap_ci(y_true, y_pred, balanced_accuracy_score)
    rec_lo, rec_hi = bootstrap_ci(
        y_true, y_pred,
        lambda yt, yp: recall_score(yt, yp, labels=[2], average=None)[0],
    )
    filas.append({
        'Modelo': nombre,
        'BalAcc': balanced_accuracy_score(y_true, y_pred),
        'BalAcc IC95 inf': bacc_lo,
        'BalAcc IC95 sup': bacc_hi,
        'Recall tb': recall_score(y_true, y_pred, labels=[2], average=None)[0],
        'Recall tb IC95 inf': rec_lo,
        'Recall tb IC95 sup': rec_hi,
    })

intervalos = pd.DataFrame(filas).sort_values('BalAcc', ascending=False)
print('Intervalos de confianza al 95 % (bootstrap estratificado):')
print(intervalos.round(4).to_string(index=False))


def mcnemar(y_true, pred_a, pred_b):
    a_ok = pred_a == y_true
    b_ok = pred_b == y_true
    n01 = np.sum(a_ok & ~b_ok)
    n10 = np.sum(~a_ok & b_ok)
    if n01 + n10 == 0:
        return np.nan, np.nan
    stat = (abs(n01 - n10) - 1) ** 2 / (n01 + n10)
    return stat, chi2.sf(stat, 1)


def tabla_mcnemar(y_eval, preds):
    nombres = list(preds)
    pares = []
    for i in range(len(nombres)):
        for j in range(i + 1, len(nombres)):
            _, p = mcnemar(y_eval, preds[nombres[i]], preds[nombres[j]])
            if p != p:
                p = 1.0
```

```
            pares.append({'Modelo A': nombres[i], 'Modelo B': nombres[j], 'p': p})
    m = len(pares)
    orden = sorted(range(m), key=lambda k: pares[k]['p'])
    anterior = 0.0
    for rango, k in enumerate(orden):
        ajustado = min(1.0, (m - rango) * pares[k]['p'])
        ajustado = max(ajustado, anterior)
        anterior = ajustado
        pares[k]['p Holm'] = ajustado
    return pares


pares = tabla_mcnemar(y_true, predicciones)
print()
print('McNemar global (OOF; p crudo y Holm):')
print(f'{"Modelo A":22s} {"Modelo B":22s} {"p":>8s} {"p Holm":>8s} {"significativo":>14s}')
for par in sorted(pares, key=lambda d: d['p']):
    sig = 'sí' if par['p Holm'] < 0.05 else 'no'
    print(f'{par["Modelo A"]:22s} {par["Modelo B"]:22s} '
          f'{par["p"]:8.4f} {par["p Holm"]:8.4f} {sig:>14s}')

mask_tb = y_true == 2
pares_tb = tabla_mcnemar(y_true[mask_tb], {n: p[mask_tb] for n, p in predicciones.items()})
print()
print('McNemar restringido a tb (OOF; p crudo y Holm):')
print(f'{"Modelo A":22s} {"Modelo B":22s} {"p":>8s} {"p Holm":>8s} {"significativo":>14s}')
for par in sorted(pares_tb, key=lambda d: d['p']):
    sig = 'sí' if par['p Holm'] < 0.05 else 'no'
    print(f'{par["Modelo A"]:22s} {par["Modelo B"]:22s} '
          f'{par["p"]:8.4f} {par["p Holm"]:8.4f} {sig:>14s}')
```

```
Intervalos de confianza al 95 % (bootstrap estratificado):
             Modelo  BalAcc  BalAcc IC95 inf  BalAcc IC95 sup  Recall tb  Recall tb IC95 inf  Recall tb
IC95 sup
                SVM  0.9494           0.9412           0.9566     0.8950              0.8725
0.9150
            XGBoost  0.9476           0.9397           0.9552     0.8788              0.8550
0.9000
Logistic Regression  0.9462           0.9383           0.9535     0.8988              0.8775
0.9188
      Random Forest  0.9085           0.8975           0.9178     0.7962              0.7650
0.8238
      Decision Tree  0.8581           0.8469           0.8685     0.7850              0.7538
0.8138

McNemar global (OOF; p crudo y Holm):
Modelo A               Modelo B                     p   p Holm  significativo
Decision Tree          XGBoost                 0.0000   0.0000             sí
SVM                    Decision Tree           0.0000   0.0000             sí
Logistic Regression    Decision Tree           0.0000   0.0000             sí
Decision Tree          Random Forest           0.0000   0.0000             sí
Random Forest          XGBoost                 0.0000   0.0000             sí
SVM                    Random Forest           0.0000   0.0000             sí
Logistic Regression    Random Forest           0.0000   0.0000             sí
Logistic Regression    XGBoost                 0.0000   0.0000             sí
Logistic Regression    SVM                     0.0001   0.0001             sí
SVM                    XGBoost                 0.0899   0.0899             no

McNemar restringido a tb (OOF; p crudo y Holm):
Modelo A               Modelo B                     p   p Holm  significativo
Logistic Regression    Random Forest           0.0000   0.0000             sí
Random Forest          XGBoost                 0.0000   0.0000             sí
Logistic Regression    Decision Tree           0.0000   0.0000             sí
SVM                    Random Forest           0.0000   0.0000             sí
SVM                    Decision Tree           0.0000   0.0000             sí
Decision Tree          XGBoost                 0.0000   0.0000             sí
Logistic Regression    XGBoost                 0.0730   0.2920             no
SVM                    XGBoost                 0.1715   0.5144             no
Decision Tree          Random Forest           0.5178   1.0000             no
Logistic Regression    SVM                     0.7277   1.0000             no
```

8. Calibración de probabilidades (tb)

In [9]:

```python
# Brier score y curva de calibración para tb sobre los scores OOF.
# Solo modelos con predict_proba: LR, DT, RF y XGBoost. SVM no expone probabilidades.

modelos_proba = [n for n in MODELOS if n != 'SVM']
y_tb = (y_true == 2).astype(int)

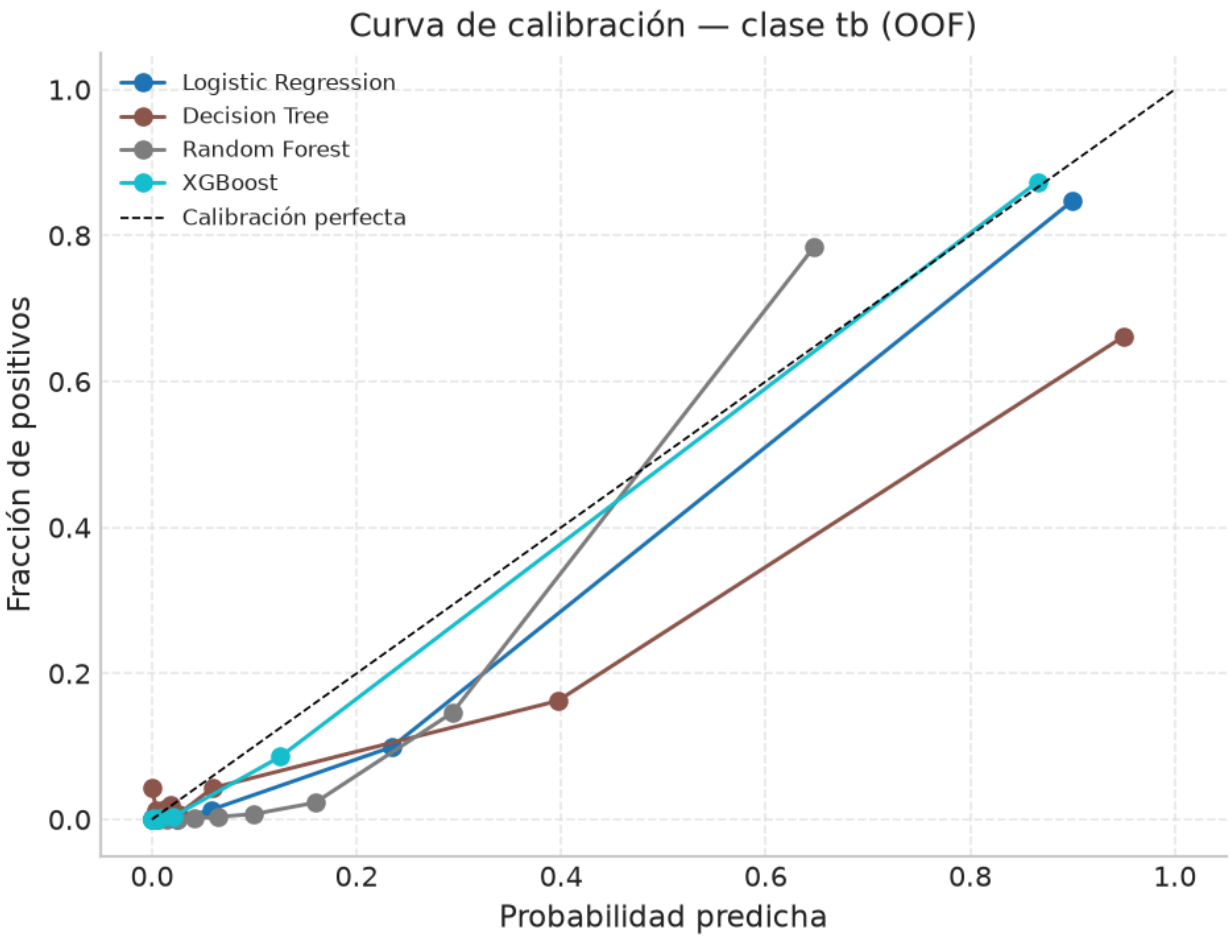
filas_cal = []
for nombre in modelos_proba:
    prob_tb = puntuaciones[nombre][:, 2]
    filas_cal.append({'Modelo': nombre, 'Brier tb': brier_score_loss(y_tb, prob_tb)})

calibracion = pd.DataFrame(filas_cal).sort_values('Brier tb')
print('Brier score para tb (menor es mejor):')
print(calibracion.round(4).to_string(index=False))
calibracion.to_csv(DATA_DIR / '09_calibracion.csv', index=False)

fig, ax = plt.subplots(figsize=(6.5, 5))
colores_modelo = dict(zip(MODELOS, colores))
for nombre in modelos_proba:
    prob_true, prob_pred = calibration_curve(
        y_tb, puntuaciones[nombre][:, 2], n_bins=10, strategy='quantile',
    )
    ax.plot(prob_pred, prob_true, 'o-', color=colores_modelo[nombre], lw=1.4, label=nombre)

ax.plot([0, 1], [0, 1], 'k--', lw=0.8, label='Calibración perfecta')
ax.set_xlabel('Probabilidad predicha', fontsize=11)
ax.set_ylabel('Fracción de positivos', fontsize=11)
ax.set_title('Curva de calibración — clase tb (OOF)', fontsize=12)
ax.legend(fontsize=8)
ax.grid(True, linestyle='--', alpha=0.4)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '09_calibracion_tb.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
Brier score para tb (menor es mejor):
              Modelo  Brier tb
             XGBoost    0.0155
Logistic Regression    0.0218
      Random Forest    0.0351
      Decision Tree    0.0659
```



9. Recomendación final

La selección del modelo se hizo en el notebook 08 con la **CV out-of-fold**. Aquí se reporta el desempeño de esos modelos sobre las mismas 8274 imágenes; el ranking es descriptivo y no se usa para reseleccionar.

In [10]:

```
# El modelo recomendado es el seleccionado por la CV OOF del 08.
ranking_oof = resultados_globales.sort_values(
    ['Balanced Acc', 'Recall tb', 'AP tb'], ascending=False,
).reset_index(drop=True)

seleccionado_cv = manifest.get('seleccionado_cv')
mejor_oof = ranking_oof.iloc[0]['Modelo']
fila_sel = ranking_oof[ranking_oof['Modelo'] == seleccionado_cv].iloc[0]

print('Ranking OOF (descriptivo):')
print(ranking_oof[['Modelo', 'Balanced Acc', 'Recall tb', 'AP tb']].round(4).to_string(index=False))
print()
print(f'Modelo recomendado (seleccionado por CV OOF en 08): {seleccionado_cv}')
print(f'  BalAcc OOF : {fila_sel["Balanced Acc"]:.4f}')
print(f'  Recall tb  : {fila_sel["Recall tb"]:.4f}')
print(f'  AP tb      : {fila_sel["AP tb"]:.4f}')
print()
print(f'Mejor OOF (descriptivo): {mejor_oof}')


def _p_holm(tabla, a, b):
    par = next((p for p in tabla if {p['Modelo A'], p['Modelo B']} == {a, b}), None)
    return None if par is None else par['p Holm']


p_global = _p_holm(pares, mejor_oof, seleccionado_cv)
p_tb = _p_holm(pares_tb, mejor_oof, seleccionado_cv)
if mejor_oof != seleccionado_cv:
```

```
    print()
    print(f'Nota: {mejor_oof} no fue el elegido por CV. No se cambia la selección.')
if p_global is not None:
    print(f'McNemar global {mejor_oof} vs {seleccionado_cv}: p Holm={p_global:.4f}')
if p_tb is not None:
    print(f'McNemar en tb  {mejor_oof} vs {seleccionado_cv}: p Holm={p_tb:.4f}')

recomendacion = {
    'politica': 'seleccion_por_cv_oof; sin hold-out',
    'modelo_recomendado': seleccionado_cv,
    'mejor_oof': mejor_oof,
    'balanced_accuracy_oof': float(fila_sel['Balanced Acc']),
    'recall_tb_oof': float(fila_sel['Recall tb']),
    'ap_tb_oof': float(fila_sel['AP tb']),
    'mejor_oof_balanced_accuracy': float(ranking_oof.iloc[0]['Balanced Acc']),
    'mcnemar_global_holm_mejor_vs_recomendado': None if p_global is None else float(p_global),
    'mcnemar_tb_holm_mejor_vs_recomendado': None if p_tb is None else float(p_tb),
}
with open(DATA_DIR / '09_recomendacion.json', 'w', encoding='utf-8') as f:
    json.dump(recomendacion, f, ensure_ascii=False, indent=2)

print()
print(f'Recomendación guardada en: {(DATA_DIR / "09_recomendacion.json").resolve()}')
```

```
Ranking OOF (descriptivo):
             Modelo  Balanced Acc  Recall tb  AP tb
                SVM        0.9494     0.8950 0.8929
            XGBoost        0.9476     0.8788 0.9596
Logistic Regression        0.9462     0.8988 0.9367
      Random Forest        0.9085     0.7962 0.8994
      Decision Tree        0.8581     0.7850 0.5824

Modelo recomendado (seleccionado por CV OOF en 08): SVM
  BalAcc OOF : 0.9494
  Recall tb  : 0.8950
  AP tb      : 0.8929

Mejor OOF (descriptivo): SVM

Recomendación guardada en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\09_recomendacion.json
```

10. Estimación OOF del procedimiento completo

Las métricas por modelo son descriptivas. La estimación menos optimista del **procedimiento completo** (selector + hiperparámetros + selección de modelo) se calculó en el notebook 08: en cada fold externo se eligió el modelo con mejor score interno y se evaluó en el fold externo. Se reporta aquí como referencia.

In [11]:
```
# Estimación OOF del procedimiento completo (sin reutilizar el ranking para reseleccionar).
proc_path = DATA_DIR / '08_procedimiento.json'
if proc_path.exists():
    with open(proc_path, encoding='utf-8') as f:
        proc = json.load(f)
    print('Modelo seleccionado en cada fold externo:', proc['selected_per_fold'])
    print()
    print('Métricas OOF del procedimiento completo:')
    for k, v in proc['metricas_oof'].items():
        print(f'  {k}: {v:.4f}')
else:
    print('No se encontró 08_procedimiento.json.')
```

```
Modelo seleccionado en cada fold externo: {'1': 'Logistic Regression', '2': 'XGBoost', '3': 'XGBoost',
'4': 'Logistic Regression', '5': 'XGBoost'}

Métricas OOF del procedimiento completo:
  Balanced Acc: 0.9479
  F1 macro: 0.9448
  Recall tb: 0.8900
  AUC-ROC macro: 0.9956
  AP tb: 0.9514
```

Conclusiones

Se evaluaron los cinco modelos con **predicciones out-of-fold sobre las 8274 imágenes**. No hay hold-out ni test separado: la estimación es la validación cruzada del notebook 08.

El accuracy global está dominado por `health` y `sick`; por eso la comparación se apoya en **balanced accuracy**, **recall de** `tb` y **AP de** `tb`. Se reportan intervalos de confianza por bootstrap, McNemar global y restringido a `tb` (con Holm) y calibración.

Limitaciones: la CV OOF no equivale a una prueba externa independiente; no hay IDs de paciente; y la selección de modelo y su estimación comparten las mismas predicciones OOF.

Distinción explícita: las métricas OOF son la estimación de la validación cruzada. El modelo final (guardado en `models/`) se entrena con las 8274 imágenes; su rendimiento en datos independientes es desconocido porque no hay test externo. La estimación del procedimiento completo está en `08_procedimiento.json`.

Fin del pipeline de modelado y evaluación.